

Group 6: Low-level Design Document

Sidharth Krishnaswamy, Nishit Oberoi, Edmond Pan, Kyle Kawasuna Saiki, Lu Maw Tun, Katia Yarkov

FrontEnd Component Design

1. Authentication

Technology:

- Framework: React
- Token Type: JWT
- Storage: HttpOnly cookie
- API calls via Axios

Internal Logic:

- State Variables:
 - email
 - password
 - authToken
 - userRole (general user vs admin)
 - errorMessage
- General Flow:
 - User submits login form
 - Send POST request to /api/auth/login
 - Receive JWT token
 - Store token
 - Redirect user to admin dashboard or general homepage

Pseudocode:

```
function handleLogin(email, password):  
    response = POST("/api/auth/login", {email, password})  
  
    if response.status == 200:  
        token = response.token  
        storeToken(token)  
        redirectToHome()  
    else:  
        showError("Invalid credentials")
```

2. Product Browsing

- State Variables:
 - productList
 - filters
 - isLoading
 - errorMessage

- Internal Logic:
 - On page load:
 - Fetch product list
 - Send GET requests to /api/products
 - Receive product data (JSON)
 - Store product list in component state
 - Render product cards
 - On filter change:
 - Capture filter input values
 - Send GET request with query parameters
 - Receive filtered product list
 - Update UI state
 - Re-render product list (no page refresh)
- Pseudocode:


```
function fetchProducts(filters == null):
    response = GET("/api/products", filters)
    setProductList(response.data)

function onFilterChange(newFilters):
    fetchProducts(newFilters)
```
- Handles initial data load & filter updates

3. Product Detail

- State Variables:
 - product → full product object returned from API
 - quantity → number user selects
 - isLoading → indicates API request in progress
 - errorMessage → for UI feedback
- Flow:
 - On component mount:
 - Extract product ID from URL
 - Send GET request to /api/products/{id}
 - Store result in state
 - Render UI
 - On quantity change:
 - Update quantity state
 - Validate it doesn't exceed stock
 - On add to cart:
 - Send POST request to /api/cart/add
 - Include product ID and quantity
 - Handle success or failure

- Pseudocode:

```
function fetchProductDetails(productId):
    set isLoading = true
    response = GET("/api/products/" + productId)

    if response.status == 200:
        set product = response.data
    else:
        set errorMessage = "Unable to load product"

    set isLoading = false

function handleQuantityChange(new Quantity):
    if newQuantity <= product.stock AND
       newQuantity > 0:
        set quantity = newQuantity
    else:
        showError("Invalid quantity")

function handleAddToCart():
    if product.stock == 0:
        return

    response = POST("/api/cart/add", {
        productId: product.id,
        quantity: quantity
    })

    if response.status == 200:
        showSuccess("Item added to cart")
    else:
        showError("Could not add item")
```

4. Shopping Cart

- State Variables:

- cartItems → array of cart item objects
- totalPrice → calculated value
- totalWeight → calculated value
- isLoading → indicates API call in progress
- errorMessage → UI feedback

- Flow:

- On component mount:
 - Send GET request to /api/cart
 - Store returned cart items in state
 - Calculate totals
- When quantity changes:
 - Validate new quantity <= stock

- Send update request to backend
- Update state
- Recalculate totals
- When item is removed:
 - Send DELETE request to backend
 - Remove item from state
 - Recalculate totals
- Pseudocode:


```
function fetchCart():
    set isLoading = true
    response = GET("/api/cart")

    if response.status == 200:
        set cartItems = response.data
        calculateTotals()
    else:
        set errorMessage = "Unable to load cart"

    set isLoading = false

function calculateTotals():
    totalPrice = 0
    totalWeight = 0

    for item in cartItems:
        totalPrice += item.price * item.quantity
        totalWeight += item.weight * item.quantity

    set totalPrice
    set totalWeight

function handleQuantityChange(productId, newQuantity):
    item = find item in cartItems

    if newQuantity > item.stockAvailable:
        showError("Quantity exceeds stock")
        return

    response = PUT("/api/cart/update", {
        productId: productId,
        quantity: newQuantity
    })

    if response.status == 200:
        update cartItems with new quantity
        calculateTotals()
```

5. Checkout

- State Variables:
 - cartItems → from backend
 - totalPrice
 - totalWeight
 - deliveryFee → returned from backend calculation
 - finalTotal → totalPrice + deliveryFee
 - selectedDeliveryDate
 - selectedTimeWindow
 - paymentDetails → card info (temporarily stored)
 - isProcessing → disables checkout buttons until payment complete
 - errorMessage
- Flow:
 - On component mount
 - GET /api/cart/summary
 - Store cart items
 - Request delivery fee calculation (based on weight)
 - Compute final total
 - On delivery selection
 - Update selectedDeliveryDate
 - Update selectedTimeWindow
 - On submit checkout
 - Validate form fields
 - Send POST /api/checkout
 - Backend:
 - Validates stock
 - Processes payment
 - Creates order
 - If success:
 - Clear cart
 - Redirect to confirmation page
 - If failure:
 - Display error to user
- Payment Processing Design
 - Use HTTPS
 - Will not store card details in localStorage
 - Immediately send payment info to backend
 - Clear payment state after submission
- Pseudocode:

```
function fetchCartSummary():
```

```

    response = GET("/api/cart/summary")

    if response.status == 200:
        set cartItems = response.data.items
        set totalPrice = response.data.totalPrice
        set totalWeight = response.data.totalWeight

function calculateDeliveryFee():
    response = POST("/api/cart/calculate-delivery", {
        totalWeight: totalWeight
    })

    if response.status == 200:
        set deliveryFee = response.data.deliveryFee
        set finalTotal = totalPrice + deliveryFee

function handleCheckout():
    if cartItems is empty:
        showError("Cart is empty")
        return

    if delivery date no selected:
        showError("Select delivery time")
        return

    if paymentDetails invalid:
        showError("Invalid payment information")
        return

    set isProcessing = true

    response = POST("/api/checkout", {
        deliveryDate,
        timeWindow,
        paymentDetails
    })

    if response.status == 200:
        clearCart()
        redirectToConfirmation()
    else:
        showError("Checkout failed")

    set isProcessing = false

```

6. Order Tracking

- State Variables:

- orderStatus → Processing / Out for delivery / Delivered
- robotLocation → {latitude, longitude}
- routePath → polyline data for map
- eta → estimated time of arrival
- pollingIntervalId → reference for clearing interval
- isDelivered → boolean for notification trigger
- errorMessage → UI error handling
- Flow:
 - On component mount:
 - Extract order ID from URL
 - Fetch initial tracking data
 - Start polling every 5-10 seconds
 - Render map
 - During polling:
 - GET /api/orders/{id}/status
 - GET /api/orders/{id}/location
 - Update state
 - Re-render map marker
 - Update ETA
 - When status is “Delivered”:
 - Stop polling
 - Trigger notification
 - Display cargo unlock instructions
- Pseudocode:


```
function fetchTrackingData(orderID):
    statusResponse = GET("/api/orders/" + orderID +
                        "/status")
    locationResponse = GET("/api/orders/" + orderID +
                          "/location")

    if statusResponse.status == 200:
        set orderStatus = statusResponse.data.status
    if locationResponse.status == 200:
        set robotLocation = locationResponse.data.coordinates
        set eta = locationResponse.data.eta
        set routePath = locationResponse.data.route
    if orderStatus == "Delivered":
        triggerArrivalNotification()
        stopPolling()

function startPolling(orderID):
    pollingIntervalId = setInterval(
        function():
            fetchTrackingData(orderID),
```

```

        5000
    )

    function stopPolling():
        clearInterval(pollingIntervalId)

    function triggerArrivalNotification():
        set isDelivered = true
        displayAlert("Your order has arrived!")

```

- Map Rendering Logic (Using Google Maps API):
 - Initialize map centered on robot location
 - Place marker at robotLocation
 - Draw polyline using routePath
 - Update marker position when state changes

7. Admin Dashboard

- State Variables:
 - currentUser (userId, name, role)
 - isAuthenticated → if user has valid session token
 - selectedView → products / inventory / orders / reports
 - products
 - orders
 - financialSummary → revenue metrics object for managers only
 - isLoading
 - errorMessage
- Flow:
 - On component mount:
 - Validate session token
 - If invalid, redirect to login
 - Fetch current user profile (with role)
 - Load default view (products)
 - When adding/editing/deleting a product:
 - Validate input fields
 - Send appropriate API request
 - Refresh product list
 - Display success/error message
 - When viewing orders:
 - GET /api/orders
 - Display order list
 - Allow filtering by status/date/customer
- Pseudocode:


```

function addProduct(productData):
    if validateProduct(productData):
        response = POST("/api/products", productData)
        if response.status == 201:
            displayAlert("Product added successfully")
            fetchproducts()
        else:
            set errorMessage = "Failed to add product"

function editProduct(productId, updatedData):
    response = PUT("/api/products/" + productId, updatedData)

    if response.status == 200:
        displayAlert("Product updated successfully")
        fetchProducts()
    else:
        set errorMessage = "Update failed"

function fetchFinancialReports():
    if currentUser.role != "manager":
        displayAlert("Access Denied")
        return
    response = GET("/api/reports/revenue")

    if response.status == 200:
        set financialSummary = response.data
    else:
        set errorMessage = "Failed to load reports"

```

BackEnd Component Design

1. Authentication Service

- State Variables:
 - userRepository
 - jwtSecret, jwtExpiry, refreshExpiry
 - cookieConfig
 - passwordHash
 - rateLimiter
- Flow: login
 - Validate request
 - Fetch user by email
 - Compare password hash
 - Valid: issue jwt

- Return user role + profile info
Logout
- Clear cookies
 - Revoke refresh token
- Pseudocode:


```

LOGIN(email, password):
    user = usersDB.find(email)

    if user not found OR password wrong:
        return "fail"

    token = makeJWT(userId, role)
    setCookie(token)

    return "ok"

    REQUIRE_AUTH(request):
        token = readCookie(request)

        if token missing OR invalid:
            return "unauthorized"

        return "ok"
      
```

2. Product Service

- State Variables: defaultPageSize, allowedSortFields, inventoryServiceClient
- Flow:
 - Read query params: search, category, minPrice, maxPrice, inStock, sort, page, pageSize
 - Validate params
 - Query DB with pagination
 - Return list and metadata (page, totalCount)
- Pseudocode:


```

GET_PRODUCTS(filters):
    return productsDB.search(filters)

GET_PRODUCT(id):
    product = productsDB.find(id)
    if not found:
        return "404"
    return product

SAVE_PRODUCT(product) [admin]:
    return productsDB.save(product)

DELETE_PRODUCT(id) [admin]:
    return productsDB.delete(id)
      
```

3. Cart and Order Service

- State Variables: cartRepository, orderRepository, productRepository, pricingCalculator, inventoryLock
- Flow:
 - Validate productId + quantity
 - Fetch product, check stock, Upsert cart_item
 - Return cart items with product snapshot fields (name, price, weight, stockAvailable)
 - Compute totals
- Transaction:
 - Load cart items
 - Validate stock again (race conditions)
 - Reserve/decrement stock
 - Create order + order_items snapshot
 - Clear cart
 - Publish event "OrderCreated(orderId)"
 - Return orderId
- Pseudocode:

```
ADD_TO_CART(userId, productId, qty):
    if qty <= 0:
        return "fail"
    cartDB.add(userId, productId, qty)
    return "ok"

GET_CART(userId):
    return cartDB.get(userId)

CHECKOUT(userId):
    items = cartDB.get(userId, paymentInfo)
    if items empty:
        return "fail"

    for each item in items:
        if productsDB.stock(item.productId) < item.qty:
            return "out_of_stock"

    totalAmount = calculateTotal(items)

    paymentResult = paymentService.charge(paymentInfo,
        totalAmount)

    if paymentResult != "success":
        return "fail"
```

```

orderId = ordersDB.create(userId, items, totalAmount)
for each item in items:
    productsDB.reduceStock(item.productId, item.qty)

cartDB.clear(userId)

commitTransaction()

return orderId

```

4. Delivery Scheduling Service

- State Variables:
 - robotRepository (status, capacity, location, battery)
 - scheduleRepository
 - maxPayloadWeight
 - minBatteryPercent
 - maxOrdersPerRun
 - delivery window rules
 - timeProvider
 - lockManager
- Flow:
 - Triggered by: OrderCreated event OR called after checkout
 - Load order details (weight, destination, priority)
 - Find candidate robots available in time window
 - Filter by capacity/battery/maintenance
 - Select robot (nearest, highest battery, least loaded, etc.)
 - Create a delivery schedule record:
 - robotId, orderId, plannedStart, plannedEnd, status “Scheduled”
 - Update order status to “Out for delivery” when dispatch begins
- Pseudocode:

```

SCHEDULE_DELIVERY(orderId) :
    order = ordersDB.get(orderId)
    robot = robotsDB.findAvailable(order.weight)

    if robot not found:
        return "no_robot"

    scheduleDB.assign(orderId, robot.id)
    return "scheduled"

```

5. Route Optimization Service

- State Variables:
 - routingMapClient
 - routeRepository

- robotTelemetryClient
- etaModel
- Flow:
 - Read origin (robot start location) + destination (customer)
 - Call routing provider for best route
 - Save ETA
- Expose endpoints:
 - GET /api/orders/{id}/route
 - GET /api/orders/{id}/eta
 - During delivery, optionally replan if robot deviates too far
- Pseudocode:


```

BUILD_ROUTE(orderId):
    robotId = scheduleDB.getRobot(orderId)
    start = robotsDB.location(robotId)
    end = ordersDB.destination(orderId)
    route = routingAPI.getRoute(start, end)

    routesDB.save(orderId, route)
    return route

GET_ROUTE(orderId):
    return routesDB.get(orderId)
      
```

Backlog

Task Name	Description	Date Created	Priority	Est. Days	T.M.	Start Date	Finish Date	Comments/ Notes
Finalize DB Schema & Tables	Create/update MySQL schema for users, products, cart, cart_items, orders, order_items, robots, schedules, routes.	02/25/26	High	3	Nishit Oberoi	3/2/26	3/4/26	Include FK constraints + indexes
Auth API Login/Logout	Implement FastAPI auth endpoints (login/logout), password validation, JWT issuance, cookie handling.	02/25/26	High	3	Sidharth Krishnaswamy	3/2/26	3/4/26	Use HttpOnly cookie as specified
Auth Middleware / Require Auth	Add backend auth guard for protected routes and role	02/25/26	High	2	Sidharth Krishnaswamy	3/5/26	3/6/26	Needed for admin dashboard

	checks (user/admin).							endpoints
Frontend Login Page	Build React login form, error handling, submit to /api/auth/login, redirect by role.	02/25/26	High	2	Katia Yarkov	3/5/26	3/6/26	Axios + JWT cookie flow
Product List API	Implement /api/products with search/filter/sort/pagination support.	02/25/26	High	4	Sidharth Krishnaswamy	3/5/26	3/10/26	Validate query params
Product Browsing UI	Build product listing page with filters/search and dynamic re-render on filter change.	02/25/26	High	4	Edmond Pan	3/7/26	3/12/26	Connect to Product List API
Product Detail API	Implement /api/products/{id} endpoint to return product details and stock info.	02/25/26	High	2	Sidharth Krishnaswamy	3/11/26	3/12/26	Return 404 for invalid product
Product Detail UI	Build product detail page with quantity validation and Add-to-Cart action.	02/25/26	High	3	Edmond Pan	3/13/26	3/17/26	Handle out-of-stock state
Cart Add/Update/Delete APIs	Implement cart endpoints for add item, update quantity, remove item, fetch cart.	02/25/26	High	4	Nishit Oberoi	3/13/26	3/18/26	Validate qty > 0 and stock bounds
Shopping Cart UI	Build cart page showing items, quantity updates, remove actions, total price/weight calculations.	02/25/26	High	4	Kyle Kawasuna Saiki	3/13/26	3/18/26	Recalculate totals on each change
Delivery Fee Calculation Logic	Implement backend delivery fee calculation based on total cart weight rules from requirements.	02/25/26	High	2	Lu Maw Tun	3/19/26	3/20/26	Match PRD pricing constraints
Cart Summary API	Implement	02/25/26	High	1	Nishit Oberoi	3/21/26	3/21/26	Used by

	/api/cart/summary including items, totalPrice, totalWeight, deliveryFee-ready data.							checkout page
Checkout API	Implement /api/checkout flow: validate cart/stock, create order, decrement inventory, clear cart.	02/25/26	High	5	Lu Maw Tun	3/22/26	3/28/26	Use transaction to avoid race issues
Mock Payment Processing	Add mock payment validation behavior for checkout (no real transactions).	02/25/26	Medium	2	Katia Yarkov	3/22/26	3/24/26	Per test plan assumptions
Checkout UI	Build checkout page with delivery date/time selection, payment fields, validation, confirmation redirect.	02/25/26	High	4	Katia Yarkov	3/22/26	3/27/26	Clear payment state after submit
Order Creation + Persistence	Implement order + order_items persistence with product snapshot data at checkout time.	02/25/26	High	2	Nishit Oberoi	3/29/26	3/30/26	Supports order history/tracking
Delivery Scheduling Service	Implement scheduling logic to assign robot based on capacity/battery/availability constraints.	02/25/26	High	5	Lu Maw Tun	3/22/26	3/28/26	Simulated robots only
Route Optimization Integration	Integrate Google Maps routing API (or mock wrapper) to build route and ETA for order delivery.	02/25/26	High	5	Lu Maw Tun	3/29/26	4/04/26	Save route + ETA in route repository
Order Status / Location APIs	Implement /api/orders/{id}/status and /api/orders/{id}/location endpoints for tracking UI polling.	02/25/26	High	3	Lu Maw Tun	4/05/26	4/07/26	Return status, coordinates, eta, route

Order Tracking UI + Map	Build order tracking page with polling, status updates, ETA, Google Maps marker + polyline route.	02/25/26	High	6	Kyle Kawasuna Saiki	3/29/26	4/06/26	Stop polling when delivered
Admin Dashboard Auth + Shell	Build admin dashboard layout, session validation, role-gated navigation (products/inventory/orders/reports).	02/25/26	High	3	Sidharth Krishnaswamy	3/29/26	3/31/26	Redirect invalid sessions to login
Admin Inventory APIs	Implement admin endpoints to view/add/update/remove stock/products.	02/25/26	High	4	Nishit Oberoi	4/01/26	4/06/26	Admin-only protected routes
Admin Inventory UI	Build admin inventory management screens for stock updates and product management.	02/25/26	High	5	Edmond Pan	4/01/26	4/07/26	Include validation + success/error feedback
Admin Orders View	Build admin view for orders and basic reporting/summary display.	02/25/26	Medium	3	Edmond Pan	4/02/26	4/06/26	Managers can review orders/reports
Concurrency / Low-Stock Handling	Add backend safeguards for simultaneous checkout on same low-stock product (locking/transaction checks).	02/25/26	High	3	Sidharth Krishnaswamy	4/01/26	4/03/26	Matches test cases in HTP
Frontend Error States + Validation Pass	Add consistent error handling/messages for invalid inputs, failed API calls, null/empty cases.	02/25/26	Medium	3	Kyle Kawasuna Saiki	4/08/26	4/10/26	Covers edge cases from test plan
Unit Test Skeletons (Backend)	Create unit tests for auth, cart, pricing, checkout, and scheduling logic.	02/25/26	Medium	4	Sidharth Krishnaswamy	4/8/26	4/13/26	Based on HL test plan categories
Feature / Integration	Test login -> browse	02/25/26	High	4	Kyle	4/8/26	4/13/26	Use mocked

Tests	-> cart -> checkout -> inventory update and admin inventory visibility interactions.				Kawasuna Saiki			external services
Cross-browser / Environment Testing	Validate key user/admin flows on Chrome and Firefox (supported browsers).	02/25/26	Medium	2	Edmond Pan	4/14/26	4/15/26	Per HLT assumptions
Final Bug Fix + Regression Pass	Resolve defects from integration/system testing and rerun core flows before submission.	02/25/26	High	4	All	4/14/26	4/19/26	Freeze scope before final packaging
Documentation + Demo Prep	Finalize LLD/HLTP updates, API notes, and demo script/screenshots for submission.	02/25/26	High	3	All	4/20/26	4/22/26	Align with CS160 deliverables